

Atividade Prática

Aula 1 — Revisão de Programação Orientada a Objetos

Técnicas de Programação II (TP2) · CST em Desenvolvimento de Software Multiplataforma · Fatec de Porto Ferreira · Prof. Vagner dos Santos

1. Identificação da atividade

Campo	Preenchimento
Aula	Aula 1 — Revisão de POO
Título da atividade	Refatoração orientada a objetos do domínio inicial do SIGA
Data de aplicação	12/08/2026
Prazo de entrega	26/08/2026
Modalidade	Individual
Duração em aula	30 minutos
Peso na avaliação	Compõe a nota de atividades práticas conforme o plano de avaliação da disciplina

2. Objetivo da atividade

Ao final desta atividade, o estudante deverá ser capaz de aplicar encapsulamento, herança e composição na modelagem do domínio inicial do sistema, demonstrando na prática os fundamentos de orientação a objetos revisados na aula.

3. Contexto

Esta atividade dá início ao Sistema de Gestão Acadêmica Simplificado (SIGA), projeto incremental da disciplina, a partir de um código inicial fornecido cuja classe Aluno expõe seus atributos publicamente, violando o encapsulamento.

4. Enunciado e etapas

Enunciado: Refatore o código inicial do SIGA aplicando os pilares da orientação a objetos e modele os relacionamentos entre as entidades do domínio.

Execute as etapas na ordem, registrando a evidência esperada de cada uma:

Etapas	Ação do estudante	Evidência esperada
1	Analisar a classe Aluno fornecida, com atributos públicos.	Identificação, por escrito, da violação do encapsulamento.
2	Refatorar a classe para atributos privados com métodos acessores validados.	Estado protegido; setter que valida os dados antes de atribuir.
3	Criar a superclasse Pessoa e derivar Aluno e Professor por herança.	Hierarquia com herança aplicada corretamente (relação "é um").
4	Modelar a classe Turma contendo Aluno por composição.	Distinção clara entre "é um" (herança) e "tem um" (composição).
5	Desenhar o diagrama de classes do domínio.	Diagrama UML consistente com o código produzido.

5. Regras de produção (fixas)

Estas regras valem para todas as atividades práticas da disciplina.

5.1 Padrão de código

- Linguagem: Java (JDK 17 ou superior).
- Identificadores (classes, atributos, métodos, variáveis) em português, seguindo o domínio do problema (ex.: Aluno, matricular, calcularMedia). Nomes claros e significativos, sem abreviações obscuras.
- Convenções Java: classes em PascalCase (Aluno, MatriculaService); métodos e atributos em camelCase (calcularMedia, nomeCompleto); constantes em UPPER_SNAKE_CASE.
- Indentação consistente (4 espaços), chaves e espaçamento padronizados. O código deve estar formatado pela IDE antes da entrega.
- Um arquivo .java por classe pública, com o nome do arquivo igual ao da classe.

5.2 Organização do projeto

- O código deve compilar e executar sem erros. Código que não compila não será avaliado nas etapas dependentes.
- Separação de responsabilidades conforme trabalhado em aula (entidades, acesso a dados, serviços, apresentação), evitando classes que acumulam múltiplas responsabilidades.
- Sem código morto, comentado em bloco ou trechos de teste esquecidos na entrega final.

5.3 Uso de ferramentas de IA

- O uso de assistentes de IA (Copilot, ChatGPT e similares) para gerar o código da atividade é PROIBIDO por padrão, pois o objetivo é o desenvolvimento da competência individual.
- A proibição só é dispensada quando o enunciado da aula indicar explicitamente o uso de IA; nesse caso, o aluno deve declarar o que foi gerado com auxílio de IA e como foi validado.
- Consultar documentação oficial da linguagem e materiais da disciplina é sempre permitido.

6. Requisitos de entrega (fixos)

- Meio de entrega: repositório Git da disciplina (padrão). O registro no SIGA/Teams, quando exigido, aponta para o repositório.
- Estrutura mínima do repositório: pasta da atividade identificada pela aula, com o código-fonte e um arquivo README.md descrevendo como compilar e executar.
- Commits com mensagens claras e em português, refletindo o progresso; evitar um único commit final com todo o trabalho.
- Entregar o código E as evidências pedidas na Seção 4 (prints, saída de execução ou diagrama, conforme a etapa).
- Em atividades em dupla, ambos os integrantes devem ter commits no repositório; a ausência de commits de um integrante impacta sua nota individual.

7. Critérios de avaliação — rubrica (fixa)

Cada critério é avaliado em três níveis. A nota da atividade resulta da soma ponderada dos critérios.

Critério	Insuficiente	Adequado	Excelente
Funcionamento e compilação	Não compila ou não executa	Compila e executa com pequenas falhas	Compila, executa e atende a todos os casos
Aplicação do conceito da aula	Conceito ausente ou incorreto	Conceito aplicado com imprecisões	Conceito aplicado corretamente e com clareza
Qualidade e organização do código	Desorganizado, sem padrão	Organizado, com padrão razoável	Coeso, bem nomeado e formatado
Cumprimento das etapas e evidências	Etapas/evidências ausentes	Maioria das etapas com evidência	Todas as etapas com evidência completa
Entrega (Git, README, commits)	Fora do padrão de entrega	Entrega no padrão com lacunas	Entrega completa e bem documentada

Pontuação: cada critério vale de 0 a 2 pontos (Insuficiente = 0; Adequado = 1; Excelente = 2), totalizando 10 pontos.

8. Integridade acadêmica e política de atraso (fixas)

- O trabalho deve ser de autoria do(s) estudante(s). Cópia entre colegas, no todo ou em parte, caracteriza plágio e zera a atividade dos envolvidos.
- Compartilhar a própria solução para cópia também constitui violação de integridade.
- O uso não declarado de IA, quando esta não foi autorizada pelo enunciado, é tratado como violação de integridade.
- Entregas após o prazo: penalidade de -20% por dia de atraso, até o limite de 2 dias; após isso, a atividade não é aceita.
- Dúvidas sobre o enunciado devem ser esclarecidas com o professor antes do prazo, preferencialmente durante a aula.